
DEVIXO SOLUTIONS

AI/ML Internship Program

Task 03:

End-to-End Machine Learning Project

Student Name:

Maryam Khan

Internship Domain:

Artificial Intelligence / Machine Learning

Submission Date:

6/09/2026

1. Introduction

In this project, a customer churn dataset is used to build predictive models that determine whether a telecom customer is likely to leave the service. The system is designed as an end-to-end machine learning solution, including data preprocessing, model training, evaluation, comparison, and deployment. The final model is integrated into a web-based dashboard using Streamlit to provide real-time predictions.

2. Objective

The objectives of this project are:

- To build and compare multiple machine learning models
 - To perform data preprocessing and feature engineering
 - To evaluate models using appropriate classification metrics
 - To select the best-performing model
 - To deploy the model using a Streamlit-based dashboard
-

3. Dataset Name

Telco Customer Churn Dataset

4. Dataset Source

The dataset is obtained from Kaggle (IBM Telco Customer Churn Dataset).

5. Dataset Description

The dataset contains customer information from a telecom company, including demographic details, account information, billing data, and services used by customers. This dataset helps in analyzing customer behavior and predicting churn, which is essential for companies to reduce customer loss and improve retention strategies.

6. Dataset Size

- Total Records: 7043

- Total Features: 20 input features
 - Target Variable: Churn (0 = No, 1 = Yes)
-

7. Columns Description

Column	Description
gender	Customer gender
SeniorCitizen	Senior citizen status
Partner	Whether customer has a partner
Dependents	Whether customer has dependents
tenure	Number of months with the company
PhoneService	Phone service subscription
MultipleLines	Multiple lines subscription
InternetService	Type of internet service
OnlineSecurity	Online security service
OnlineBackup	Online backup service
DeviceProtection	Device protection
TechSupport	Technical support
StreamingTV	Streaming TV service
StreamingMovies	Streaming movies service
Contract	Contract type
PaperlessBilling	Paperless billing status
PaymentMethod	Payment method
MonthlyCharges	Monthly charges
TotalCharges	Total charges

Column	Description
Churn	Target variable

8. Project Structure

Task_03_ML/

```
|
|— dataset/
|   |— Telco_Customer_Charm.csv
|
|— notebook/
|   |— Task_03_ML.ipynb
|— Task_03_ML.pdf
|
|— app.py
|— best_churn_model.joblib
|— scaler.joblib
|— model_columns.joblib
|— README.md
```

9. Methodology

The following steps were followed:

1. Data loading
 2. Data preprocessing
 3. Feature engineering
 4. Feature scaling
 5. Model training
 6. Model evaluation
 7. Model comparison
 8. Hyperparameter tuning
 9. Model deployment
-

10. Data Loading

The dataset was loaded using the Pandas library:

```
df = pd.read_csv("Telco_Customer_Churn.csv")
```

```
df = pd.read_csv("Telco_Customer_Churn.csv")

print(df.head(5))
```

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	
0	7590-VHVEG	Female	0	Yes	No	1	No	
1	5575-GNVDE	Male	0	No	No	34	Yes	
2	3668-QPYBK	Male	0	No	No	2	Yes	
3	7795-CFOCW	Male	0	No	No	45	No	
4	9237-HQITU	Female	0	No	No	2	Yes	

	MultipleLines	InternetService	OnlineSecurity	...	DeviceProtection	
0	No phone service	DSL	No	...	No	
1	No	DSL	Yes	...	Yes	
2	No	DSL	Yes	...	No	
3	No phone service	DSL	Yes	...	Yes	
4	No	Fiber optic	No	...	No	

	TechSupport	StreamingTV	StreamingMovies	Contract	PaperlessBilling	
0	No	No	No	Month-to-month	Yes	
1	No	No	No	One year	No	
2	No	No	No	Month-to-month	Yes	
3	Yes	No	No	One year	No	
4	No	No	No	Month-to-month	Yes	

	PaymentMethod	MonthlyCharges	TotalCharges	Churn
0	Electronic check	29.85	29.85	No
1	Mailed check	56.95	1889.5	No
2	Mailed check	53.85	108.15	Yes
3	Bank transfer (automatic)	42.30	1840.75	No
4	Electronic check	70.70	151.65	Yes

11. Data Preprocessing

The dataset was preprocessed to improve data quality:

- Missing values in TotalCharges were handled by converting it to numeric and filling null values

```
df['TotalCharges'] = pd.to_numeric(df['TotalCharges'], errors='coerce')

print("Missing TotalCharges after conversion:", df['TotalCharges'].isnull().sum())
print(df[df['TotalCharges'].isnull()][['tenure', 'MonthlyCharges', 'TotalCharges']])
df['TotalCharges'] = df['TotalCharges'].fillna(0)

print("Missing after fill:", df['TotalCharges'].isnull().sum())
print(df['TotalCharges'].dtype)
```

```
... Missing TotalCharges after conversion: 11
   tenure  MonthlyCharges  TotalCharges
488      0           52.55           NaN
753      0           20.25           NaN
936      0           80.85           NaN
1082     0           25.75           NaN
1340     0           56.05           NaN
3331     0           19.85           NaN
3826     0           25.35           NaN
4380     0           20.00           NaN
5218     0           19.70           NaN
6670     0           73.35           NaN
6754     0           61.90           NaN
Missing after fill: 0
float64
```

- Duplicate records were checked and removed

```
print("Duplicate rows before:", df.duplicated().sum())

df = df.drop_duplicates()

print("Shape after removing duplicates:", df.shape)

Duplicate rows before: 0
Shape after removing duplicates: (7043, 21)
```

- New features such as AvgMonthlyCharge were created
- Target variable (Churn) was converted into binary format

```
# Drop identifier column
df = df.drop(columns=['customerID'])

# Encode target variable
df['Churn'] = df['Churn'].map({'Yes': 1, 'No': 0})

# Feature 1: Total number of subscribed services
service_cols = ['PhoneService', 'MultipleLines', 'OnlineSecurity', 'OnlineBackup',
                'DeviceProtection', 'TechSupport', 'StreamingTV', 'StreamingMovies']

def count_services(row):
    return sum(1 for col in service_cols if row[col] == 'Yes')

df['TotalServices'] = df.apply(count_services, axis=1)

# Feature 2: Average monthly spend
df['AvgMonthlySpend'] = df['TotalCharges'] / df['tenure'].replace(0, 1)

print(df[['TotalServices', 'AvgMonthlySpend', 'Churn']].head())
print(df.shape)
print(df['Churn'].value_counts())
```

```
✓ 0.5s

TotalServices  AvgMonthlySpend  Churn
0              1          29.850000      0
1              3          55.573529      0
2              3          54.075000      1
3              3          40.905556      0
4              1          75.825000      1
(7043, 22)
Churn
0      5174
1      1869
Name: count, dtype: int64
```

- Categorical features were encoded

```

cat_cols = df.select_dtypes(include='object').columns.tolist()
print("Categorical columns to encode:", cat_cols)

df_encoded = pd.get_dummies(df, columns=cat_cols, drop_first=True)

print("Shape after encoding:", df_encoded.shape)

# Correlation with target
corr = df_encoded.corr()['Churn'].sort_values(ascending=False)
print("Top correlated features with Churn:\n", corr.head(5))
print("\nLeast correlated features with Churn:\n", corr.tail(5))

```

✓ 0.1s

```

Categorical columns to encode: ['gender', 'Partner', 'Dependents', 'PhoneService', 'MultipleLines', 'InternetService', 'OnlineSecurity', 'OnlineBackup']
Shape after encoding: (7043, 33)
Top correlated features with Churn:
Churn          1.000000
InternetService_Fiber optic  0.308020
PaymentMethod_Electronic check  0.301919
MonthlyCharges  0.193356
AvgMonthlySpend  0.193301
Name: Churn, dtype: float64

Least correlated features with Churn:
DeviceProtection_No internet service -0.227890
StreamingMovies_No internet service -0.227890
TechSupport_No internet service -0.227890
Contract_Two year -0.302253
tenure -0.352229
Name: Churn, dtype: float64

```

- Numerical features were scaled using StandardScaler

```

from sklearn.preprocessing import StandardScaler

numeric_cols = ['tenure', 'MonthlyCharges', 'TotalCharges', 'TotalServices', 'AvgMonthlySpend']

scaler = StandardScaler()
df_encoded[numeric_cols] = scaler.fit_transform(df_encoded[numeric_cols])

print(df_encoded[numeric_cols].describe())
print(df_encoded.shape)
# saving data for part2
df_encoded.to_csv("telco_processed.csv", index=False)
print("Saved telco_processed.csv")

```

✓ 3.6s

	tenure	MonthlyCharges	TotalCharges	TotalServices
count	7.043000e+03	7.043000e+03	7.043000e+03	7.043000e+03
mean	-2.421273e-17	-6.406285e-17	-3.783239e-17	-5.044319e-17
std	1.000071e+00	1.000071e+00	1.000071e+00	1.000071e+00
min	-1.318165e+00	-1.545860e+00	-1.005780e+00	-1.630990e+00
25%	-9.516817e-01	-9.725399e-01	-8.299464e-01	-1.145997e+00
50%	-1.372744e-01	1.857327e-01	-3.905282e-01	-1.760106e-01
75%	9.214551e-01	8.338335e-01	6.648034e-01	7.939757e-01
max	1.613701e+00	1.794352e+00	2.825806e+00	2.248955e+00

	AvgMonthlySpend
count	7.043000e+03
mean	7.465592e-17
std	1.000071e+00
min	-2.137475e+00
25%	-9.597171e-01
50%	1.850696e-01
75%	8.416645e-01
max	1.873292e+00

(7043, 33)
Saved telco_processed.csv

- Dataset was split into training and testing sets

```

from sklearn.model_selection import train_test_split

X = df_encoded.drop(columns=['Churn'])
y = df_encoded['Churn']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print("X_train shape:", X_train.shape)
print("X_test shape:", X_test.shape)
print("y_train distribution:\n", y_train.value_counts(normalize=True))
print("y_test distribution:\n", y_test.value_counts(normalize=True))

```

✓ 0.1s

```

X_train shape: (5634, 32)
X_test shape: (1409, 32)
y_train distribution:
Churn
0    0.734647
1    0.265353
Name: proportion, dtype: float64
y_test distribution:
Churn
0    0.734564
1    0.265436
Name: proportion, dtype: float64

```

12. Model Development

Each model was trained using the training dataset. The following machine learning models were developed:

- Decision Tree

```

from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

dt_model = DecisionTreeClassifier(random_state=42)
dt_model.fit(X_train, y_train)

y_pred_dt = dt_model.predict(X_test)

print("Decision Tree Accuracy:", accuracy_score(y_test, y_pred_dt))
print("\nClassification Report:\n", classification_report(y_test, y_pred_dt))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_dt))

```

✓ 0.4s

```

Decision Tree Accuracy: 0.7416607523066004

Classification Report:

```

	precision	recall	f1-score	support
0	0.83	0.82	0.82	1035
1	0.51	0.52	0.52	374
accuracy			0.74	1409
macro avg	0.67	0.67	0.67	1409
weighted avg	0.74	0.74	0.74	1409

```

Confusion Matrix:
[[849 186]
 [178 196]]

```


- Random Forest

```
from sklearn.ensemble import RandomForestClassifier

rf_model = RandomForestClassifier(random_state=42, class_weight='balanced')
rf_model.fit(X_train, y_train)

y_pred_rf = rf_model.predict(X_test)

print("Random Forest Accuracy:", accuracy_score(y_test, y_pred_rf))
print("\nClassification Report:\n", classification_report(y_test, y_pred_rf))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_rf))
```

✓ 2.0s

```
Random Forest Accuracy: 0.7679205110007097

Classification Report:
      precision    recall  f1-score   support

     0       0.86      0.82      0.84      1035
     1       0.55      0.64      0.59       374

 accuracy      0.77      0.77      0.77      1409
 macro avg       0.71      0.73      0.72      1409
weighted avg       0.78      0.77      0.77      1409

Confusion Matrix:
[[844 191]
 [136 238]]
```

- Support Vector Machine (SVM)

```
from sklearn.svm import SVC

svm_model = SVC(random_state=42, class_weight='balanced')
svm_model.fit(X_train, y_train)

y_pred_svm = svm_model.predict(X_test)

print("SVM Accuracy:", accuracy_score(y_test, y_pred_svm))
print("\nClassification Report:\n", classification_report(y_test, y_pred_svm))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_svm))
```

✓ 3.8s

```
SVM Accuracy: 0.7501774308019872

Classification Report:
      precision    recall  f1-score   support

     0       0.90      0.74      0.81      1035
     1       0.52      0.78      0.62       374

 accuracy      0.75      0.75      0.75      1409
 macro avg       0.71      0.76      0.72      1409
weighted avg       0.80      0.75      0.76      1409

Confusion Matrix:
[[764 271]
 [ 81 293]]
```

- Gradient Boosting

```
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.utils.class_weight import compute_sample_weight

gb_model = GradientBoostingClassifier(random_state=42)

sample_weights = compute_sample_weight(class_weight='balanced', y=y_train)
gb_model.fit(X_train, y_train, sample_weight=sample_weights)

y_pred_gb = gb_model.predict(X_test)

print("Gradient Boosting Accuracy:", accuracy_score(y_test, y_pred_gb))
print("\nClassification Report:\n", classification_report(y_test, y_pred_gb))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_gb))
```

✓ 2.5s

Gradient Boosting Accuracy: 0.7459190915542938

Classification Report:

	precision	recall	f1-score	support
0	0.91	0.72	0.81	1035
1	0.51	0.81	0.63	374
accuracy			0.75	1409
macro avg	0.71	0.77	0.72	1409
weighted avg	0.81	0.75	0.76	1409

Confusion Matrix:

```
[[747 288]
 [ 70 304]]
```

13. Model Evaluation

Models were evaluated using classification metrics:

- Accuracy
- Precision
- Recall
- F1 Score
- Confusion Matrix

These metrics were used to assess the performance of each model.

14. Model Comparison

Model	Performance	Advantages	Limitations
Decision Tree	Moderate	Simple and interpretable	Prone to overfitting
Random Forest	High	Robust and accurate	Higher computational cost
SVM	Moderate	Strong performance in classification	Sensitive to scaling
Gradient Boosting	High	High accuracy and performance	Slower training

```

from sklearn.model_selection import cross_val_score

models = {
    "Decision Tree": dt_model,
    "Random Forest": rf_model,
    "SVM": svm_model,
    "Gradient Boosting": gb_model
}

for name, model in models.items():
    scores = cross_val_score(model, X_train, y_train, cv=5, scoring='f1')
    print(f"{name}: Mean F1 = {scores.mean():.4f}, Std = {scores.std():.4f}")

```

✓ 25.9s

```

Decision Tree: Mean F1 = 0.5048, Std = 0.0046
Random Forest: Mean F1 = 0.6092, Std = 0.0179
SVM: Mean F1 = 0.6195, Std = 0.0201
Gradient Boosting: Mean F1 = 0.5754, Std = 0.0377

```

15. Hyperparameter Tuning

Hyperparameter tuning was performed using GridSearchCV with cross-validation to optimize model performance. This process helped in selecting the best-performing model.

```

from sklearn.model_selection import GridSearchCV

# Decision Tree
dt_params = {
    'max_depth': [3, 5, 7, 10, None],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4]
}
dt_grid = GridSearchCV(DecisionTreeClassifier(random_state=42), dt_params, cv=5, scoring='f1', n_jobs=-1)
dt_grid.fit(X_train, y_train)
print("Decision Tree best params:", dt_grid.best_params_)
print("Decision Tree best F1:", dt_grid.best_score_)

```

✓ 12.4s

Decision Tree best params: {'max_depth': 10, 'min_samples_leaf': 1, 'min_samples_split': 5}
 Decision Tree best F1: 0.5334052620907104

```

rf_params = {
    'n_estimators': [100, 200],
    'max_depth': [5, 10, None],
    'min_samples_split': [2, 5],
    'min_samples_leaf': [1, 2]
}
rf_grid = GridSearchCV(
    RandomForestClassifier(random_state=42, class_weight='balanced'),
    rf_params, cv=5, scoring='f1', n_jobs=-1
)
rf_grid.fit(X_train, y_train)
print("Random Forest best params:", rf_grid.best_params_)
print("Random Forest best F1:", rf_grid.best_score_)

```

✓ 1m 13.2s

Random Forest best params: {'max_depth': 10, 'min_samples_leaf': 2, 'min_samples_split': 2, 'n_estimators': 200}
 Random Forest best F1: 0.634084345041378

```

svm_params = {
    'C': [0.1, 1, 10],
    'kernel': ['rbf', 'linear'],
    'gamma': ['scale', 'auto']
}
svm_grid = GridSearchCV(
    SVC(random_state=42, class_weight='balanced'),
    svm_params, cv=5, scoring='f1', n_jobs=-1
)
svm_grid.fit(X_train, y_train)
print("SVM best params:", svm_grid.best_params_)
print("SVM best F1:", svm_grid.best_score_)

```

✓ 1m 15.4s

SVM best params: {'C': 0.1, 'gamma': 'scale', 'kernel': 'rbf'}
 SVM best F1: 0.6258361018613793

```

gb_params = {
    'n_estimators': [100, 200],
    'learning_rate': [0.05, 0.1],
    'max_depth': [3, 5]
}
gb_grid = GridSearchCV(
    GradientBoostingClassifier(random_state=42),
    gb_params, cv=5, scoring='f1', n_jobs=-1
)
gb_grid.fit(X_train, y_train, sample_weight=sample_weights)
print("Gradient Boosting best params:", gb_grid.best_params_)
print("Gradient Boosting best F1:", gb_grid.best_score_)

```

✓ 49.6s

Gradient Boosting best params: {'learning_rate': 0.05, 'max_depth': 3, 'n_estimators': 100}
 Gradient Boosting best F1: 0.7782958503178032

16. Model Deployment

The trained model, scaler, and feature columns were saved using:

```
joblib.dump(rf_best, "best_churn_model.joblib")
```

```
joblib.dump(scaler, "scaler.joblib")
```

```
joblib.dump(list(X_train.columns), "model_columns.joblib")
```

The model was integrated into a Streamlit application to provide real-time predictions.

```
import joblib
import pickle

# Save with Joblib
joblib.dump(rf_best, "best_churn_model.joblib")

# Save with Pickle
with open("best_churn_model.pkl", "wb") as f:
    pickle.dump(rf_best, f)

joblib.dump(scaler, "scaler.joblib")
joblib.dump(list(X_train.columns), "model_columns.joblib")

print("Model, scaler, and columns saved successfully.")
```

✓ 0.3s

Model, scaler, and columns saved successfully.

17. Web Application

A web-based dashboard was developed using Streamlit.

Features:

- Interactive user interface
- Input of customer details
- Real-time prediction
- Churn probability score and classification

Churn Predictor

Maryam Khan

Task 03 · AI/ML Internship · Devixo Solutions

Model

Random Forest Classifier

Tuned via GridSearchCV, 5-fold CV

Dataset

Telco Customer Churn

7,043 customers · 21 attributes

This app is a demo interface for a churn-prediction model trained end-to-end: data cleaning, feature engineering, model comparison, and hyperparameter tuning. See the accompanying notebook and report for full methodology.

Developed by Maryam Khan · AI/ML Internship Project

Customer Churn Prediction

Estimate the likelihood that a telecom customer will churn, based on their account, billing, and service details.

18. Input Features

The model uses the following features:

- Gender
- SeniorCitizen
- Partner
- Dependents
- Tenure

- PhoneService
- MultipleLines
- InternetService
- OnlineSecurity
- OnlineBackup
- DeviceProtection
- TechSupport
- StreamingTV
- StreamingMovies
- Contract
- PaperlessBilling
- PaymentMethod
- MonthlyCharges
- TotalCharges

Customer Profile

Demographics

Gender

Male

Senior Citizen

No

Partner

Yes

Dependents

Yes

Account

Tenure (months)

12

Contract

Month-to-month

Paperless Billing

Yes

Payment Method

Electronic check

Billing

Monthly Charges (\$)

70.00

Total Charges (\$)

800.00

Services

Phone Service

Yes

Internet Service

DSL

Online Backup

Yes

Tech Support

Yes

Multiple Lines

Yes

Online Security

Yes

Device Protection

Yes

Streaming TV

Yes

Streaming Movies

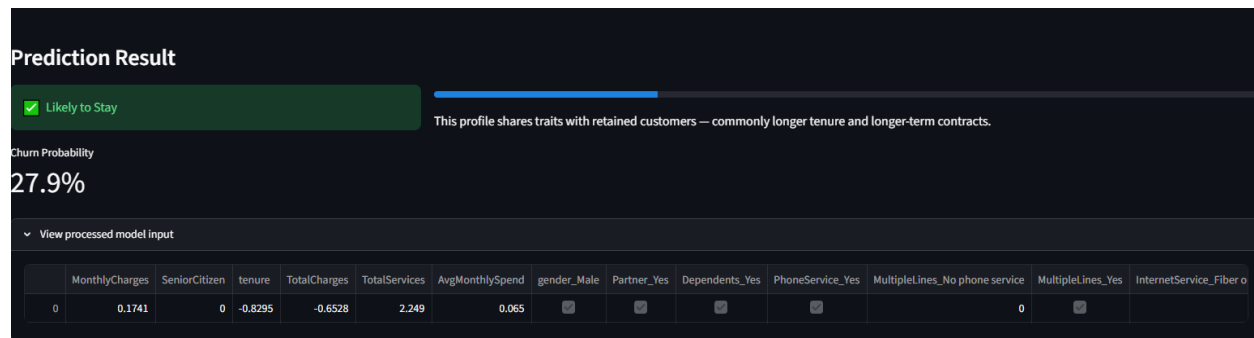
Yes

Predict Churn

19. Output

The system provides:

- Prediction
- Churn probability score



20. Key Highlights

Complete machine learning pipeline

Multiple model comparison

Hyperparameter tuning and cross-validation

Feature engineering and scaling

Deployment using Streamlit

Interactive dashboard

Real-world dataset

21. Conclusion

This project successfully implemented a complete machine learning pipeline for customer churn prediction. Multiple models were trained and evaluated, and the Random Forest model performed the best after hyperparameter tuning. The project demonstrates how machine learning can be applied in real-world business scenarios to reduce customer churn and improve retention strategies. The final system was deployed using Streamlit, allowing users to input customer data and receive real-time predictions.

22. GitHub Repository

https://github.com/maryamkhan68/Customer_Churn.git